ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART XV HANDS-ON IMPLEMENTATION GUIDE

Progressive Labs: From Bare Cluster to Full Enterprise AI Platform

Volume 15 of 16 Implementation Series | Edition 2025-2026

LAB GUIDE OVERVIEW

Progressive Implementation Path

These labs progress from basic cluster setup to a full production AI platform. Each lab builds on the previous. Complete all prerequisites before starting each lab. Estimated total time: 40-60 hours for all 10 labs.

Lab	Title	Hours	Prerequisites	Key Skills
1	Bootstrap Production Cluster	4-6	Cloud account, kubectl, helm	kubeadm/managed K8s, Cilium, ArgoCD
2	Deploy and Scale Microservice	3-4	Lab 1	Deployment, HPA, Ingress, cert-manager, Prometheus
3	Zero Trust Networking	4-5	Lab 2	Kyverno, Istio Ambient, NetworkPolicy, Falco
4	GitOps with ArgoCD	3-4	Lab 1	ApplicationSet, Argo Rollouts, canary deployment
5	Secure Secret Management	3-4	Lab 1	Vault HA, ESO, dynamic secrets, secret rotation
6	Storage, Snapshots, DR	4-5	Lab 1	CSI, StorageClass, VolumeSnapshot, Velero
7	GPU Workload + vLLM	4-6	Lab 1, GPU node	GPU Operator, MIG, vLLM, DCGM metrics
8	Full RAG Pipeline	5-6	Labs 6,7	Qdrant, embedding Job, RAG API, OTel traces
9	Agentic AI with Temporal	5-6	Labs 7,8	Temporal, agent worker, KEDA, MCP server
10	Production Hardening	4-5	Labs 1-9	CIS benchmark, Trivy cluster scan, IR playbook

LAB 1

Bootstrap a Production Cluster

* Install kubeadm on 3 control plane VMs (or use managed EKS/GKE/AKS)
* Deploy Cilium CNI with kube-proxy replacement and WireGuard encryption
* Configure OIDC (Dex or Keycloak) for cluster authentication
* Install ArgoCD (HA mode) and bootstrap the App-of-Apps pattern
* Install: cert-manager, Vault (dev mode), Prometheus stack, Loki, Falco
* Verify: kubectl cluster-info, kubectl top nodes, argocd app list

Verification Commands:

```
kubectl get nodes -o wide cilium status argocd app list kubectl get pods -A | grep -v Running
```

LAB 2

Deploy and Scale a Microservice

* Create a Deployment with resource requests, security context, and health probes
* Expose via Service and configure Ingress with TLS via cert-manager ACME
* Create HPA targeting 70% CPU utilisation (minReplicas=2, maxReplicas=10)
* Create Prometheus ServiceMonitor for the application metrics endpoint
* Import or create Grafana dashboard showing request rate, latency, error rate
* Load test with k6 and observe HPA scaling decisions in real time

Verification Commands:

```
kubectl describe hpa myapp kubectl get ingress curl -k https://myapp.example.com/health k6 run load-test.js
```

LAB 3

Zero Trust Networking

* Apply default-deny NetworkPolicy to production namespace
* Enable Istio Ambient Mesh: kubectl label ns production istio.io/dataplane-mode=ambient
* Apply Kyverno ClusterPolicies: require-nonroot, require-readonly-rootfs, require-seccomp
* Verify mTLS is enforced: istioctl authn tls-check
* Test NetworkPolicy: verify blocked traffic between non-allowed namespaces
* Deploy Falco and trigger a test alert by spawning a shell in a container

Verification Commands:

```
kubectl exec -n test -- sh # Falco should alert istioctl authn tls-check kubectl get netpol -A
```

LAB 4

GitOps with ArgoCD

* Create an ApplicationSet deploying to dev, staging, and production from one template
* Implement Argo Rollouts canary: 5% -> 20% -> 50% -> 100% with Prometheus analysis
* Simulate a bad deployment: cause errors in canary, observe automatic rollback
* Enable selfHeal: true and prune: true; make a manual kubectl change and watch it revert
* Review ArgoCD application history and practice rollback to a previous revision

Verification Commands:

```
argocd app get myapp-production kubectl argo rollouts status myapp kubectl argo rollouts undo myapp
```

LAB 5

Secure Secret Management

* Install Vault in HA mode with Raft backend and auto-unseal (cloud KMS)
* Configure Kubernetes auth method in Vault
* Create dynamic PostgreSQL credentials with 1-hour TTL via Vault database engine
* Install External Secrets Operator and create ExternalSecret syncing from Vault
* Demonstrate secret rotation: rotate PostgreSQL password; Pod gets new creds without restart
* Verify: no secrets in environment variables or log output

Verification Commands:

```
vault status vault read database/creds/myapp-role kubectl get externalsecret -A kubectl describe secret myapp-db-creds
```

LAB 6

Storage, Snapshots, and Disaster Recovery

* Install cloud CSI driver and create gp3 StorageClass with Retain reclaim policy
* Deploy PostgreSQL StatefulSet with PVC; write test data
* Install VolumeSnapshot CRDs and CSI snapshotter; take a snapshot
* Restore snapshot to new PVC in staging namespace; verify data integrity
* Expand the original PVC by 50% without downtime
* Install Velero; create scheduled backup; simulate disaster; restore from backup

Verification Commands:

```
kubectl get pvc -A kubectl get volumesnapshot -A velero backup describe my-backup velero restore create --from-backup my-backup
```

LAB 7

GPU Workload and vLLM Deployment

* Install NVIDIA GPU Operator on a GPU node (g5.xlarge or similar)
* Verify GPU resource is schedulable: kubectl describe node | grep nvidia.com/gpu
* Configure MIG partitioning on A100 (if available): 3x 1g.10gb instances
* Deploy vLLM with Qwen2.5-1.5B-Instruct (small model for cost-effective testing)
* Load test with 10 concurrent users and observe DCGM GPU utilisation in Grafana
* Implement KEDA ScaledObject scaling vLLM replicas on request queue depth

Verification Commands:

```
kubectl describe node gpu-node | grep -i nvidia curl http://vllm:8000/v1/models kubectl top pods -n ai-serving
```

LAB 8

Full RAG Pipeline

* Deploy Qdrant StatefulSet with 500GB NVMe PVC
* Run embedding generation Job: ingest 10,000 documents from S3 into Qdrant
* Deploy RAG API server (FastAPI) with OTel instrumentation
* Wire RAG API -> Qdrant (vector search) -> vLLM (generation) -> response
* Run 20 test queries and verify retrieved context + generated responses
* Open Grafana: view traces in Tempo showing full RAG request path

Verification Commands:

```
kubectl get pods -n ai-platform curl http://rag-api/query -d '{"question": "What is our refund policy?"}' hubble observe --namespace ai-platform
```

LAB 9

Agentic AI with Temporal

* Install Temporal on Kubernetes with PostgreSQL backend and web UI
* Write a simple research agent workflow in Python SDK (web search + LLM synthesis)
* Deploy agent worker Deployment (2 replicas); apply KEDA ScaledObject on Kafka task queue
* Submit 5 workflows via Temporal CLI and observe execution in the Temporal web UI
* Kill a worker mid-execution; observe Temporal automatically resume on another worker
* Deploy an MCP server with web search tool; update agent to use MCP for tool calls

Verification Commands:

```
temporal workflow list temporal workflow describe -w WORKFLOW_ID kubectl delete pod temporal-worker-xyz # Test fault tolerance temporal workflow list # Verify workflow resumed
```

LAB 10

Production Hardening and Compliance

* Run kube-bench CIS Kubernetes Benchmark; remediate all Level 1 findings
* Run trivy k8s --report summary cluster; remediate CRITICAL image CVEs
* Enforce Pod Security Standards Restricted on all production namespaces
* Configure Falco with PagerDuty integration; test with a simulated attack (shell exec)
* Run RBAC audit: find all cluster-admin bindings; remove unnecessary ones

* Document and test the incident response playbook: detect, contain, forensics, remediate

Verification Commands:

```
kube-bench run --targets master,node trivy k8s --report summary cluster kubectl auth can-i
--list --as=system:serviceaccount:prod:myapp-sa
```